IN PARTNERSHIP WITH PLYMOUTH UNIVERSITY

Name: SAP Prabhashana (Group 80)	
Student Reference Number: 10967100	

Module Code: PUSL 2021	Module Name: Computing Group Project
Coursework Title: Interim Report Submission	
Deadline Date: 19/02/2026	Member of staff responsible for coursework: Mr. Anton Jayakodi
Programme: BSc (Hons) Data Science , BSc (Hons) Artificial Intelligence	

Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook.

Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts.

Name	Student ID
SAP Prabhashana	10967100
Rubasinghe D D	10967104
YUS De Yapa	10967122
NPMMD Pathirana	10967063
RRI Dissanayaka	10967081
AVJA Amarasekera	10967076
PGT Sewmini	10967111

We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.

Signed on behalf of the group:

Individual assignment: ***I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.***

Signed :

Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence.

I *have used/not used translation software.

If used, please state name of software……

Overall mark _____% **Assessors Initials _____** **Date – 19/02/2026**

Sign of the Supervisor

TABLE OF CONTENTS

CHAPTER 01 – INTRODUCTION 6

1.1 Introduction 6
1.2 Problem Definition 6
1.3 Project Objectives 7
Objectives for LinkAble Vision 7
Objectives for LinkAble Connect 8

CHAPTER 02 – SYSTEM ANALYSIS 9

2.1 Fact Gathering Techniques 9
2.2 Existing System 9
2.3 Drawbacks of the Existing System 11

CHAPTER 03 – REQUIREMENTS SPECIFICATION 12

3.1 Overview 12
3.2 Stakeholders, users, and external systems 12
3.3 Functional requirements 12
3.3.1 Shared platform FRs 13
3.3.2 LinkAble Vision FRs 13
3.3.3 LinkAble Connect FRs 13
3.4 Non-functional requirements 14
3.5 Acceptance criteria and traceability 14

CHAPTER 04 – FEASIBILITY STUDY 15

4.1 Introduction 15
4.2 Feasibility Dimensions (TELOS summary) 15
4.3 Technical Feasibility 16
4.4 Economic Feasibility 16
4.5 Operational Feasibility 16
4.6 Legal and Ethical Feasibility 17
4.7 Conclusion 17

CHAPTER 5 – SYSTEM ARCHITECTURE 18

5.1 Use Case Diagram 18
5.2 Class Diagram of the Proposed System 19
5.3 ER Diagram 20
5.4 High-Level Architectural Diagram 21
5.5 Networking Diagram 21

CHAPTER 06 – DEVELOPMENT TOOLS & TECHNOLOGIES 22

6.1 Introduction 22
6.2 Overall engineering approach 22
6.3 Tools and technologies for LinkAble Vision 22
Mobile application layer 22
Native mobile components 22
Navigation and location 23
6.4 Tools and technologies for LinkAble Connect 23
Conversion and sharing 23
6.5 Collaboration, quality, and project tooling 23

Version control and teamwork *23*
Testing *23*
Build and debug *24*
6.6 Summary table 24

CHAPTER 07 – IMPLEMENTATION PROGRESS 25

7.1 Introduction 25
7.2 Overall Progress Summary 25
7.3 LinkAble Vision – Implementation Progress 25
7.3.1 Implemented: Camera pipeline and controlled sampling 25
7.3.2 Implemented: On-device inference integration 26
7.3.3 Implemented: Alert engine (non-spam, safety-first speech) 26
7.3.4 Implemented: Sensitivity modes and localization-ready outputs 26
7.3.5 Implemented: Debug and evaluation tooling 27
7.3.6 In progress: Navigation session and SOS integration 27
7.4 LinkAble Connect – Implementation Progress 28
7.4.1 Implemented: Android keyboard (IME) core workflow 28
7.4.2 Implemented: Token format enforcement and local preview 28
7.4.3 Implemented: On-device suggestion concept (baseline) 29
7.4.4 Implemented: Telegram inline bot and sign-strip rendering 29
7.4.5 In progress: iOS keyboard extension and production hardening 29
7.5 Integration Progress and Interfaces 29
7.6 Testing and Validation Progress 29
7.7 Challenges Encountered and Solutions Applied 30
7.8 Remaining Work Before Final Delivery 30
7.9 Interim Conclusion 30

CHAPTER 08 – DISCUSSION 31

REFERENCES 32

TEAM PLAN & RESPONSIBILITY MATRIX 33

List of Tables

Table 1. Existing approaches and how LinkAble differs 10
Table 2. Functional requirements 12
Table 3. Non-functional requirements 14
Table 4. TELOS feasibility assessment 15
Table 5. Technologies used in LinkAble 24
Table 6. High-level progress by application 25
Table 7. Integration status for shared elements 30
Table 8. Responsibility Matrix 33

List of Figures

Figure 1. Use Case Diagram 18
Figure 2. Class Diagram 19
Figure 3. ER Diagram 20
Figure 4. High-Level Architectural Diagram 21
Figure 5. Networking Diagram 21
Figure 6. Code snippets of categorizing obstacles 27
Figure 7. Current status of the Linkable connect with a new addition of preview bar 28

Chapter 01 – Introduction

1.1 Introduction

The assistive technology platform LinkAble provides users with two essential solutions which help people with disabilities to navigate safely and communicate effectively through their platform. Users need to use multiple applications because current tools only provide solutions for one aspect of their problem. The two mobile applications LinkAble Vision and LinkAble Connect work together to provide users a unified experience through their shared configuration services and content assets, which include lexicons and phrasebooks.

LinkAble Vision provides a navigation solution which helps blind and low-vision users who need audio feedback during their walking activities. The system provides users with walking directions which use spoken instructions for navigation while enabling them to identify obstacles through their device's phone camera. Users need to understand the obstacle alerts because they must understand the alerts which should produce only intermittently active sounds based on the specification requirements. People who experience excessive speech interruptions during their movement activities will experience decreased safety.

LinkAble Connect functions as the communication platform which enables users to create Sri Lankan Sign Language (SLSL) messages through a mobile keyboard (Android IME and iOS keyboard extension) which it provides together with a feature that shows their sign sequences in a shareable format through Telegram inline bot. The Connect application solves two primary issues: it enables users to enter sign language through sign language typing and it enables users to share their sign language messages. The application establishes specific sign tokens which enable users to generate sign-strip images through the process of combining multiple sign icons into a single long strip which the system will divide into multiple strips when the length exceeds certain limits.

Both applications are designed with accessibility, privacy, and real-world constraints in mind. Vision should continue local obstacle alerts even when network conditions prevent routing updates and Connect should allow offline typing and preview even when bot conversion is unavailable. In addition, the platform is planned to support multilingual accessibility (English, Sinhala, and Tamil) and to be evaluated with measurable indicators such as navigation accuracy, alert usability, conversion correctness, and user satisfaction in outdoor and messaging scenarios during pilot testing.

1.2 Problem Definition

People who have visual impairment, hearing loss, and non-verbal disabilities experience various challenges which restrict their ability to live independently, ensure personal safety, and engage with society. The project proposal demonstrates the extent of the problem by presenting international data on vision impairment and disabling hearing loss while demonstrating how these conditions create difficulties for people to move and communicate which leads to their social isolation and decreased life chances and increased danger in public spaces. Sri Lanka and similar locations face increased difficulties because they lack affordable solutions and suitable resources. People encounter multiple challenges because many assistive tools need expensive costs and active internet connections and their design focuses primarily on Western markets while public areas lack accessible signs and local sign-language databases remain restricted. The process allows users to operate public areas through which they depend on guide

assistance to travel public spaces while they need help to decode spoken content and to communicate through common dialogue.

The system faces an essential problem because it consists of multiple disconnected parts. Navigation assistants create routes but they lack effective obstacle detection while communication tools provide transcription and text-to-speech functions but they cannot handle sign language development and sending sign messages through basic platforms. The LinkAble solution divides into two main components which function as equal mainstays while Vision delivers voice-guided outdoor navigation and on-device obstacle detection and SOS location dispatch and Connect enables users to create signs through token-based typing which contains on-device recommendation systems and Telegram inline bot for secure unknown word management through sign creation.

Therefore, the problem addressed by LinkAble can be stated as follows:

1) For navigation: an affordable, mobile-first solution is needed that combines voice-guided walking navigation with on-device obstacle alerts that remain useful in dynamic environments without producing continuously noisy audio.

2) For communication: a reliable sign-first messaging method for SLSL is needed that enables fast composition, deterministic representation (tokens), and practical sharing through a widely used platform, with safe handling of unknowns and clear behavior under weak networks.

3) For the overall ecosystem: an integrated approach is needed that supports both domains with consistent usability, shared configuration, and measurable requirements for performance, security, reliability, and accessibility.

1.3 Project Objectives

The objectives below define what the LinkAble project will achieve by the end of development. They keep equal focus on both applications and remain feasible and testable within the MVP scope.

Platform-level objectives (shared)

- To deliver two accessibility applications (Vision and Connect) with consistent usability and ethical design.
- To maintain shared configuration and content assets (lexicon/phrasebook) with versioning to ensure consistent conversions and outputs over system releases.
- To satisfy measurable non-functional requirements for reliability (graceful offline behavior), security/privacy (safe handling and opt-in behavior where applicable), and performance targets appropriate for real use.

Objectives for LinkAble Vision

- To provide voice-guided turn-by-turn walking navigation with rerouting when the user deviates from the intended route.
- To implement on-device obstacle detection from the camera feed and generate short, timely audio alerts that are understandable and not continuously noisy.
- To provide an SOS feature that shares the user’s location with predefined contacts and clearly reports success or failure.

Objectives for LinkAble Connect

- To provide a system-level SLSL keyboard (Android IME and iOS extension) that allows sign-first composition using sign-icon keys.
- To insert deterministic sign tokens into chat inputs while showing a local preview bar that stays in sync with edits.
- To provide an opt-in Telegram inline bot that converts tokens to Sinhala and Sinhala to sign-strip images, with explicit unknown-word handling and deterministic rendering via icon stitching and splitting.

Chapter 02 – System Analysis

2.1 Fact Gathering Techniques

System analysis for LinkAble used several techniques to keep decisions evidence-based. First, we performed document analysis by reviewing the Proposal and the Functional & Technical Specification (F&TS) to extract scope, user roles, functional behaviors, and measurable acceptance criteria for both LinkAble Vision and LinkAble Connect. The F&TS defines the platform as two equal-focus applications and specifies key behaviors such as spoken navigation and obstacle alerts for Vision, and token-based keyboard input plus Telegram inline-bot conversions for Connect. Second, we held stakeholder review discussions with supervisors and peers to validate that the MVP boundaries are realistic and that the in-scope / out-of-scope decisions are defensible for an interim checkpoint. Third, we carried out comparative analysis of existing tools (navigation assistants, object-detection aids, and sign/messaging workflows) to understand how the current world handles similar problems and where gaps remain for accessibility, Sri Lankan localization, and SLSL support. Finally, we used prototyping observations: by running initial builds (camera pipeline + alert tests for Vision, and keyboard/bot tests for Connect), we observed latency, usability friction, and failure modes under weak network and missing permissions. These observations match the F&TS emphasis on clear errors and on graceful offline behavior.

2.2 Existing System

The LinkAble system analysis utilized multiple methods which supported the project decisions through direct assessment of documented evidence. We performed document analysis through our examination of the Proposal and the Functional & Technical Specification (F&TS) which allowed us to identify the project scope and user roles and functional requirements and measurable acceptance criteria for both LinkAble Vision and LinkAble Connect. The F&TS defines the platform as two equal-focus applications and specifies key behaviors such as spoken navigation and obstacle alerts for Vision and token-based keyboard input plus Telegram inline-bot conversions for Connect. We conducted stakeholder review discussions with our supervisors and peers in order to determine whether the defined MVP boundaries could exist as real limits and to confirm that our project scope decisions reached appropriate validation for the checkpoint assessment. We conducted a comparative study of existing tools which include navigation assistants and object-detection aids and sign/messaging workflows to investigate how current systems solve similar issues and to identify accessibility gaps which exist between Sri Lankan localization and SLSL support. The initial builds which we used for prototyping tests which included the camera pipeline and alert tests for Vision and keyboard/bot tests for Connect enabled us to observe system performance issues through testing within restricted network conditions and when permissions were not available. The F&TS document requires the system to display clear error messages and to maintain operational capabilities during offline periods.

Deaf and HoH people use text chat and stickers and video calls to communicate because they cannot hear and speak. The reason people need to create text for sign language is that sign language cannot be represented through text and people need to create video content for their specific needs when using low bandwidth or protecting their privacy or sending messages that need to be read later. The current sticker-based sign sets fail to provide correct SLSL grammatical structures because they lack proper sign inventory which needs to be maintained according to their established standards. LinkAble Connect solves the input issue through its

system-wide SLSL keyboard which works as an Android IME and iOS extension that provides preset sign tokens and displays a typing preview feature. The output problem solution uses a Telegram inline bot which requires user consent to transform between three types of content: tokens and Sinhala text and sign-strip images. The solution requires users to mark content which they do not know while LinkAble Connect establishes secure methods for content which users need to identify as unknown.

The current system exhibits weaknesses that affect both operational consistency and organizational control across its entire framework. The different applications require different vocabulary and settings and update procedures which leads to unstable user experience during extended periods. LinkAble enables administrators to manage core services which handle data for lexicons and phrasebooks through version control while enabling identical conversion and configuration processes to function across different software releases.

Table 1. Existing approaches and how LinkAble differs

Area	Typical existing approach	Key limitation	LinkAble approach
Vision	Voice navigation apps	No obstacle awareness	Navigation + on-device obstacle alerts
Vision	Camera detection apps	Noisy/latency/cloud reliance	Timely, non-spam alerts; offline-safe behavior
Connect	Text chat / captions	Not sign-first	Sign keyboard + preview + suggestions
Connect	Video signing	Bandwidth + async limits	Token output + bot rendering into sign strips

2.3 Drawbacks of the Existing System

The present disconnected method brings about operational issues together with safety problems. A user who is correctly "on route" during transit can still experience unexpected crashes with obstacles which are not included in map data. Users who test camera detection systems will hear the same voice announcements which repeat "person, person, person" while this information hides essential threats and creates mental strain that decreases user confidence and system usage. The system includes two types of connectivity dangers because it depends on cloud services for detection and for handling rerouting events which may lead to hidden operational failures. LinkAble's requirements solve these problems through their need for permission verification together with their system which warns users about weak GPS signals and their system which provides detailed recovery instructions and their system which continues to show local obstacles even when routing information cannot be updated.

Existing messaging systems create communication barriers for Deaf users because they require Deaf users to express their thoughts through writing which takes more time and results in communication errors. The use of non-standard sign stickers creates confusion because translation programs generate results that show high confidence yet deliver incorrect output. Automated conversion systems become highly error-prone when they lack deterministic encoding methods. The token format and bot functionality of LinkAble Connect create a system gap which enables users to work offline by typing and previewing content while system conversion occurs when users choose to start the process which displays errors for incorrect input and shows flags for unrecognized terms. The majority of unstructured systems do not provide controlled phrasebook management systems which leads to inaccurate results over time. LinkAble established its main services to operate with controlled lexicon and phrasebook systems which use versioning to guarantee that identical token sequences will generate identical mapped results when the same version is active.

Chapter 03 – Requirements Specification

3.1 Overview

The Proposal together with the Functional and Technical Specification document F&TS serves as the basis for establishing requirements which guide development of the interim build. The requirements are divided into three categories which include shared platform requirements LinkAble Vision requirements and LinkAble Connect requirements. Vision and Connect exist as two distinct applications which share only their assets that include lexicon and phrasebook and configuration, and they share their quality constraints which include security and reliability and accessibility. The purpose is to make every requirement verifiable through a demonstration or a test or documented proof.

3.2 Stakeholders, users, and external systems

Users are: (1) blind/low-vision users who rely on spoken guidance while walking, and (2) Deaf/Hard-of-Hearing or non-verbal users who compose Sri Lankan Sign Language (SLSL) messages. The system has three emergency contact groups which include SOS recipients and administrators who control shared resources. The system requires a routing provider to enable navigation through its interface and it needs to access device services which include GPS and compass and camera and notification and audio functions and Telegram for inline-bot sharing. The system must degrade safely when dependencies fail, and Connect must enforce strict parsing and explicit unknown handling throughout.

3.3 Functional requirements

The F&TS function lists for Vision and Connect define functional requirements (FRs) which include the Vision requirement that obstacle alerts must be timely and not continuously noisy, and the Connect requirement that the keyboard outputs tokens and preview in scope.

Table 2. Functional requirements

Area	ID	Requirement statement	Verification
Shared	SH-FR-01	Provide onboarding/settings to enable Vision and Connect independently.	Screens + demo
Shared	SH-FR-02	Maintain versioned lexicon/phrasebook assets used by conversions and labels.	Version check
Shared	SH-FR-03	Capture metrics/events (errors, latency, drop counts) with opt-in consent.	Logs + toggle
Vision	LV-FR-01	Voice-guided turn-by-turn walking navigation to a chosen destination.	Walk demo
Vision	LV-FR-02	Detect deviation and provide rerouting guidance.	Deviate test
Vision	LV-FR-03	On-device obstacle detection from camera feed with concise, non-spam audio alerts.	Live alerts
Vision	LV-FR-04	SOS sends current location to predefined contacts and reports success/failure.	SOS demo

Connect	LC-FR-01	SLSL keyboard (Android IME / iOS extension) inserts deterministic sign tokens in any text field.	Type test
Connect	LC-FR-02	On-device next-sign suggestions (n-gram/frequency) without network.	Suggest demo
Connect	LC-FR-03	Local preview of sign icons stays synchronized with edits.	Edit test
Connect	LC-FR-04	Telegram inline bot converts tokens↔Sinhala and returns rendered sign-strip images.	Bot demo
Connect	LC-FR-05	Rendering stitches icons and splits long outputs deterministically.	Repeatability
Connect	LC-FR-06	Unknown tokens/words are explicitly marked; no guessing.	Unknown test

3.3.1 Shared platform FRs

SH-FR-01 prevents cross-feature interference by allowing Vision and Connect to be configured and tested separately (language, speech settings, privacy consent). SH-FR-02 establishes governance procedures for shared content which require versioned assets to maintain consistent conversion processes while updating content. SH-FR-03 provides visibility into performance claims through the measurement of navigation update delays and obstacle inference latency and bot response time.

3.3.2 LinkAble Vision FRs

LV-FR-01: The user selects a destination and starts a walking session. The system speaks step instructions (direction + distance) and updates them as the user progresses.

LV-FR-02: If the user leaves the intended path, the system detects deviation and issues rerouting guidance without restarting.

LV-FR-03: While active, the camera feed is sampled and processed on-device to detect common obstacles. Alerts must be short, understandable, and must follow anti-noise rules so they are not continuously noisy in practice. Safety-critical hazards have priority (e.g., approaching vehicles) and may interrupt lower-priority messages. The system must support offline-safe behavior: if routing updates fail, obstacle alerts continue and the app provides a clear routing unavailable state.

LV-FR-04: SOS shares current location with predefined contacts; if sharing fails, the user receives a clear error and retry guidance.

3.3.3 LinkAble Connect FRs

LC-FR-01: The user enables the SLSL keyboard and inputs sign keys in any compatible text field. Output is deterministic tokens so bot parsing is reliable and safe throughout.

LC-FR-02: The keyboard provides next-sign suggestions using an on-device model to reduce typing effort; suggestions must not require internet.

LC-FR-03: The preview bar visualizes the current sign sequence; it must remain correct under insert/delete, cursor moves, and replacements.

LC-FR-04: When the user triggers sharing, a Telegram inline bot converts tokens→Sinhala text and Sinhala→sign-strip image(s), returning results with safe errors.

LC-FR-05: Sign-strip rendering stitches icons in order and splits long results at defined boundaries; identical input yields identical output.

LC-FR-06: Unknown tokens/words must be explicitly marked; the system must not guess an alternative sign because it could change meaning.

3.4 Non-functional requirements

Non-functional requirements (NFRs) apply to both applications, with app-specific emphasis.

Table 3. Non-functional requirements

Category	NFR ID	Requirement	Applies to
Performance	NFR-P-01	Priority speech: critical alerts are delivered promptly without long queues.	Vision + Connect
Performance	NFR-P-02	Real-time obstacle usability via frame sampling/backpressure and latency.	Vision
Reliability	NFR-R-01	Safe degradation under weak/no internet: Vision keeps obstacle alerts; Connect keeps typing/preview.	Vision + Connect
Reliability	NFR-R-02	Permission and service failures show actionable recovery, not crashes.	Vision + Connect
Security	NFR-S-01	TLS for external calls; strict input validation and payload limits (routing/bot).	Vision + Connect
Privacy	NFR-PR-01	SOS and bot actions are user-triggered; analytics is opt-in and anonymized.	Vision + Connect
Usability	NFR-U-01	Accessibility-first UI and multilingual-ready message keys.	Vision + Connect
Compatibility	NFR-C-01	Android + iOS support; Connect obeys OS keyboard extension constraints.	Vision + Connect

3.5 Acceptance criteria and traceability

Each requirement is validated by one of three evidence types: (1) scenario demonstrations (walk navigation, obstacle alerts, keyboard typing, bot conversion), (2) automated tests for deterministic logic (alert gating, render split rules, token parsing), and (3) runtime logs/metrics showing timing, suppression counts, failures, and recovery steps. This ensures the interim deliverable is measurable and avoids untestable functionality.

Chapter 04 – Feasibility Study

4.1 Introduction

This feasibility study evaluates whether LinkAble can be delivered to a strong interim level within the time, skills, and constraints of a university computing group project. LinkAble includes two distinct mobile applications which are LinkAble Vision that provides navigation and obstacle detection for blind and low-vision users and LinkAble Connect that combines SLSL sign-token keyboard with Telegram inline bot to enable sign message sharing. The two applications require separate assessments of their core risks because their different accessibility domains need operational testing to confirm all shared feasibility factors which include deployment and security and testing.

.

4.2 Feasibility Dimensions (TELOS summary)

Table 4. TELOS feasibility assessment

Dimension	Feasibility status	Key reasoning	Main risks	Mitigation
Technical	High	Proven mobile stacks exist for on-device ML, routing APIs, keyboard IME/extensions, and Python bot rendering	Latency/noise in Vision alerts; OS restrictions for keyboards; bot reliability	Backpressure + alert gating; limit keyboard scope; strict token parsing + rate limits
Economic	Medium–High	Low infrastructure needs for interim; on-device processing reduces cloud cost	Hosting cost for bot/service; paid map API quotas	Use free tiers; cache; lightweight hosting; measure and optimize usage
Legal/Ethical	Medium	Sensitive user groups; privacy and safety matter	Location misuse; wrong sign conversions; unsafe navigation outputs	Explicit consent; opt-in sharing; deterministic tokens; safe error handling
Operational	High	Clear user workflows; offline-safe behavior reduces dependency failures	Real-world onboarding complexity	Guided setup; permissions checks; simple defaults
Schedule	Medium	Deliverables can be staged by modules	Integration delays between teams	Interface contracts; weekly integration checkpoints

4.3 Technical Feasibility

LinkAble Vision:

The Vision MVP depends on two technical pillars which include voice-guided navigation using a routing provider and device GPS/compass and on-device obstacle detection with safe audio alerts. Mobile platforms enable both of these features because they provide camera access and optimized inference runtimes such as TensorFlow Lite and low-latency event pipelines. The main feasibility risk is not can we detect objects, but can we deliver usable alerts. The real-world environment produces excessive speech because too many detections occur. The engineered alert logic controls risk through its various components which include sampling control and frame dropping and cooldowns and priority and deduplication and summary-style announcements which replace repetitive labels. The technical risk emerges from changing lighting conditions and motion blur but the interim solution minimizes this risk because it targets common obstacles through confidence threshold adjustments and robust fallback systems which maintain obstacle alerts during navigation updates.

LinkAble Connect:

The technical implementation of Connect becomes possible through Android's provision of custom keyboard support through IME while iOS permits users to access keyboard extensions but imposes specific limitations. The keyboard must reliably output deterministic sign tokens and keep a preview synchronized with edits, which is achievable through predictable token structures combined with internal cursor tracking and limited UI elements which include simple keys and suggestion bar and preview strip. The Telegram inline bot is feasible using Python because deterministic parsing and sign-strip rendering become easy to accomplish through standard image libraries. The primary technical threats to the project include three different components which are (1) iOS extension memory/time limitations and (2) token parsing problems and (3) problems associated with processing lengthy messages and unfamiliar mapping systems. The keyboard maintains its lightweight design while enforcing strict token format requirements and deterministic output boundaries and explicit identification of unknown elements instead of making assumptions about them.

4.4 Economic Feasibility

The interim build maintains its financial viability because Vision obstacle detection operates on devices, which decreases expenses for server operations. The probable expenses for the Telegram bot hosting and route API access remain optional. The primary economic threat stems from the implementation of paid routing tiers and existing quotas. The organization can reduce expenses by choosing a service provider which offers a no-cost tier while implementing proper caching and controlling route requests and assessing system usage throughout testing to prevent unanticipated traffic increases.

4.5 Operational Feasibility

The two applications maintain high operational feasibility because they match actual user requirements: Vision enables users to start navigation through spoken directions and Connect functions as a keyboard that shares through Telegram within existing messaging systems. The main operational hurdles involve setup procedures which include obtaining permissions and enabling keyboard functionality and selecting language and voice settings and maintaining

services during periods of poor internet connectivity. The system can be managed through guided onboarding which provides permission prompts with visible instructions and offline-safe behavior: Vision maintains obstacle alerts when users lack routing access while Connect provides internet-free access to typing and preview features and users choose whether to use bot conversion.

4.6 Legal and Ethical Feasibility

The project processes confidential information which includes users' location data and their camera feeds and their patterns of using accessibility features. The ethical feasibility of the project requires implementation of privacy-by-design combined with safe failure mechanisms which include obtaining explicit consent for SOS sharing and opting-in to bot usage and retaining only essential data and displaying clear notifications that the system helps users but does not substitute for professional mobility training. The Connect system produces incorrect output which leads to misinterpreted results because it outputs incorrect sign output. The use of deterministic tokens with explicit unknown handling enables users to convert results safely because it prevents them from making incorrect assumptions about their confidence level.

4.7 Conclusion

Overall feasibility is strong for an interim deliverable. The highest risks—Vision alert usability, keyboard OS constraints, and bot reliability—are real but manageable through careful scope control, deterministic design, strict validation, and staged integration with measurable testing.

Chapter 5 – System Architecture

5.1 Use Case Diagram

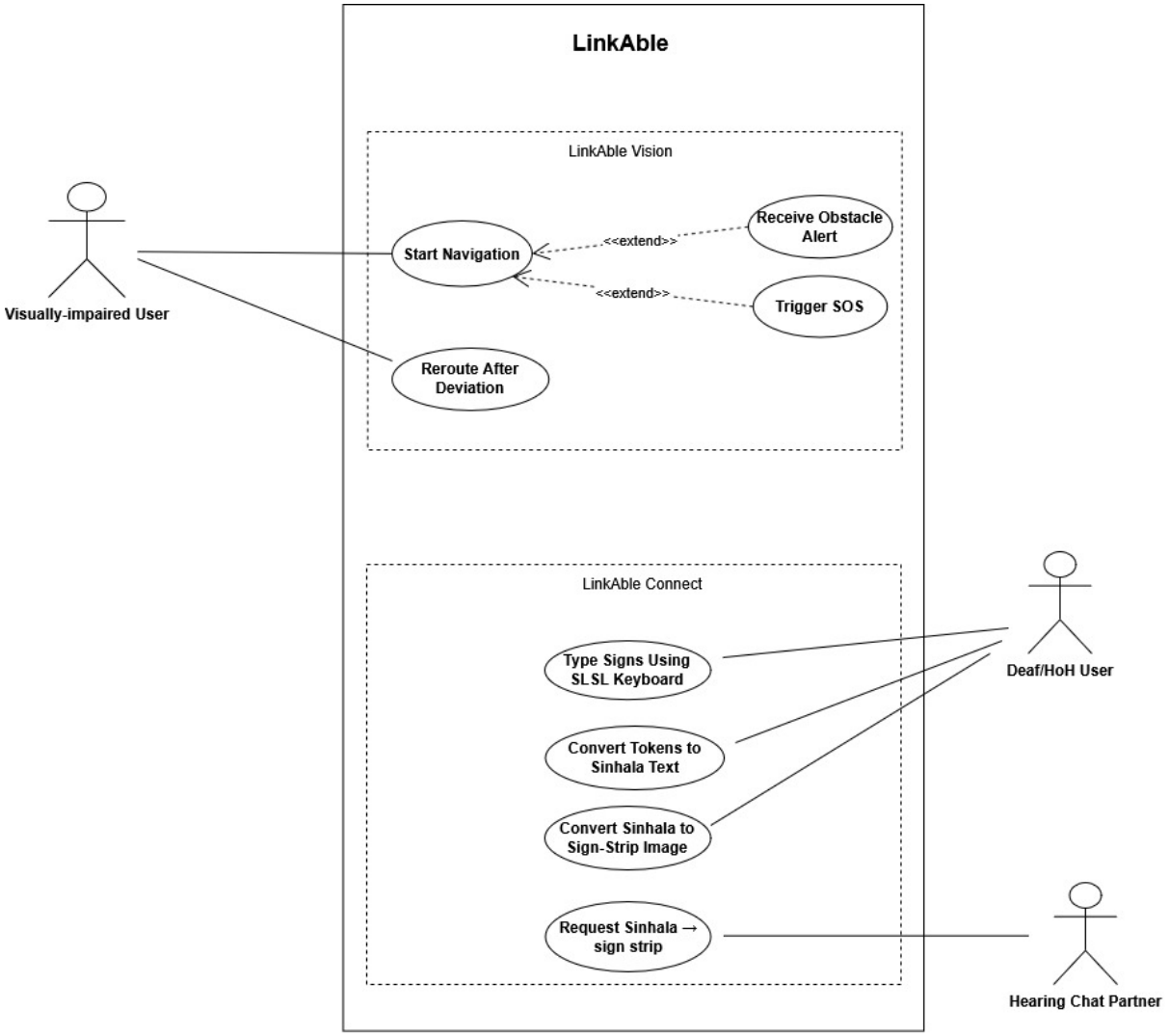

Figure 1. Use Case Diagram

Note:

- "**Trigger SOS**" extends "**Start Navigation**" for diagram simplicity but is available anytime during Vision app use (per LV-FR-05).
- The diagram uses separate "**LinkAble Vision**" and "**LinkAble Connect**" boundaries to reflect the system's dual-focus architecture.

5.2 Class Diagram of the Proposed System

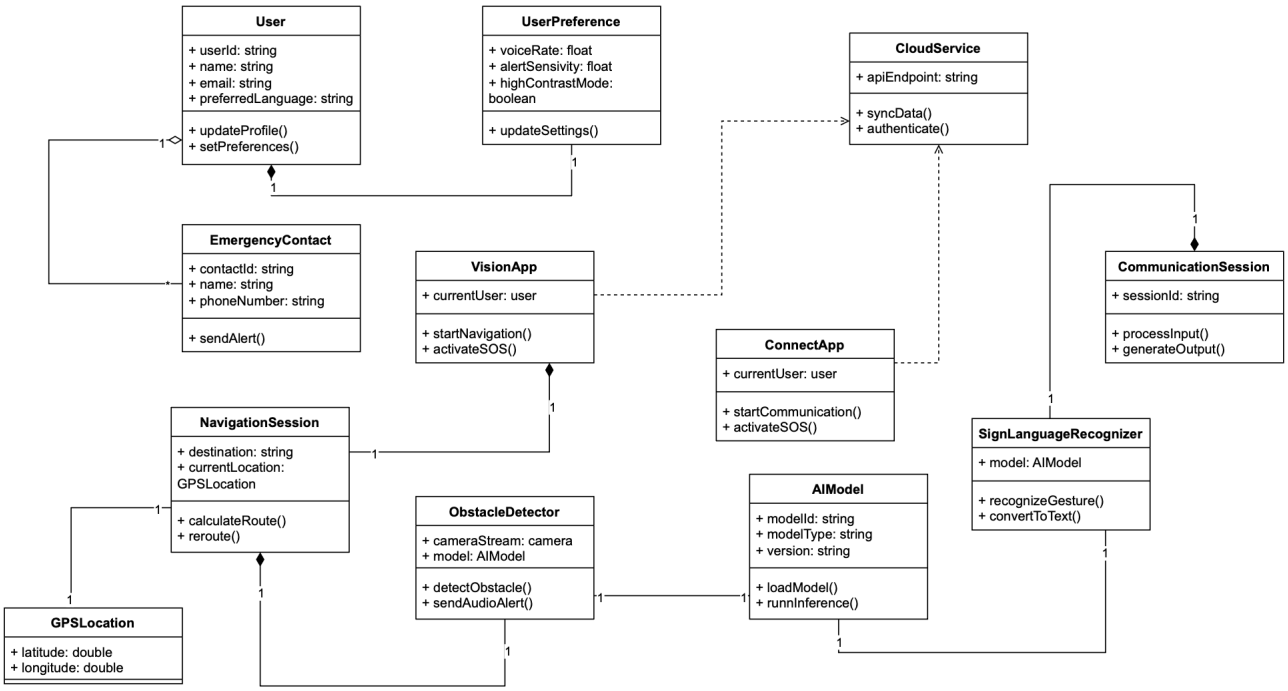

Figure 2. Class Diagram

5.3 ER Diagram

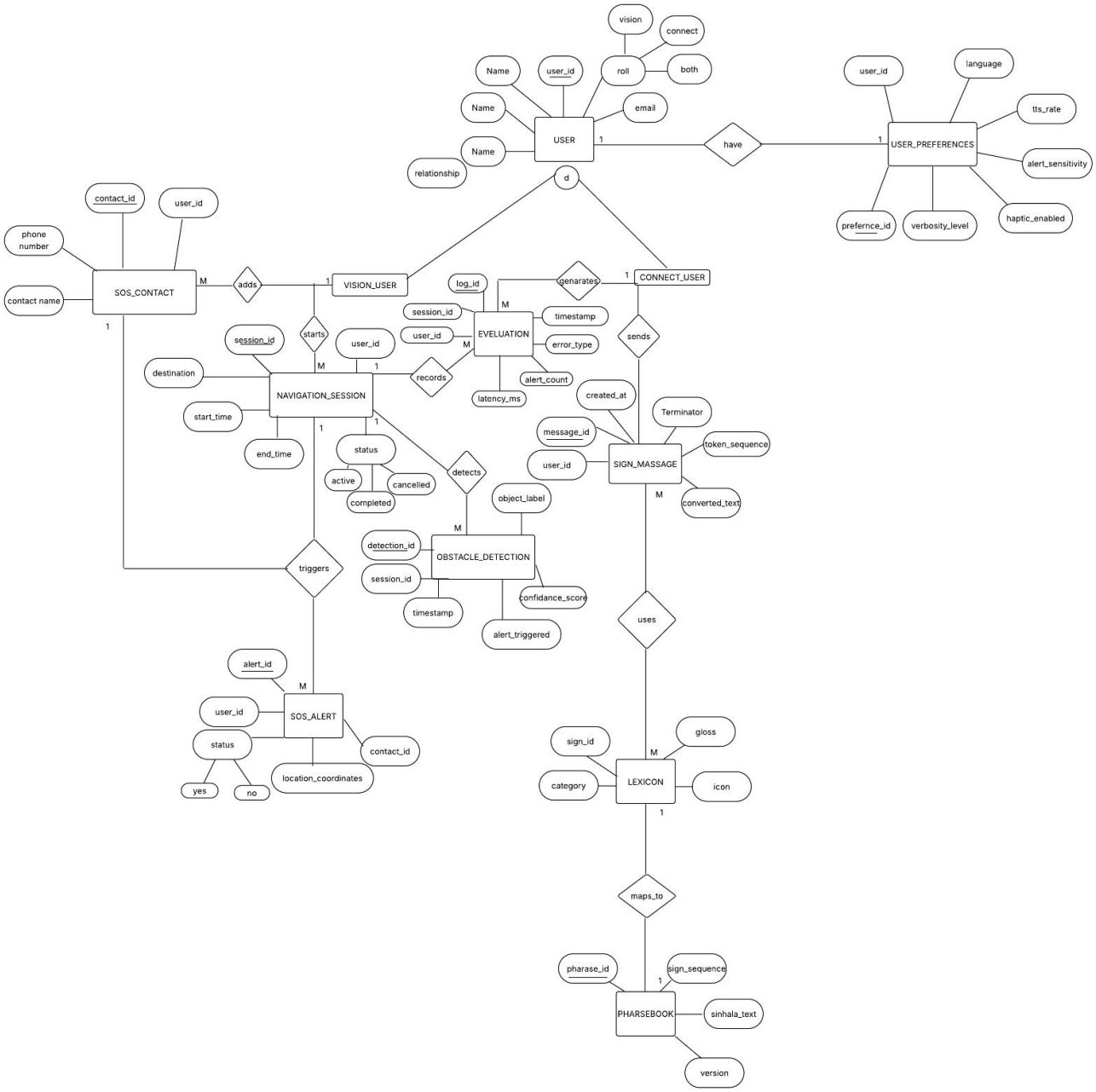

Figure 3. ER Diagram

5.4 High-Level Architectural Diagram

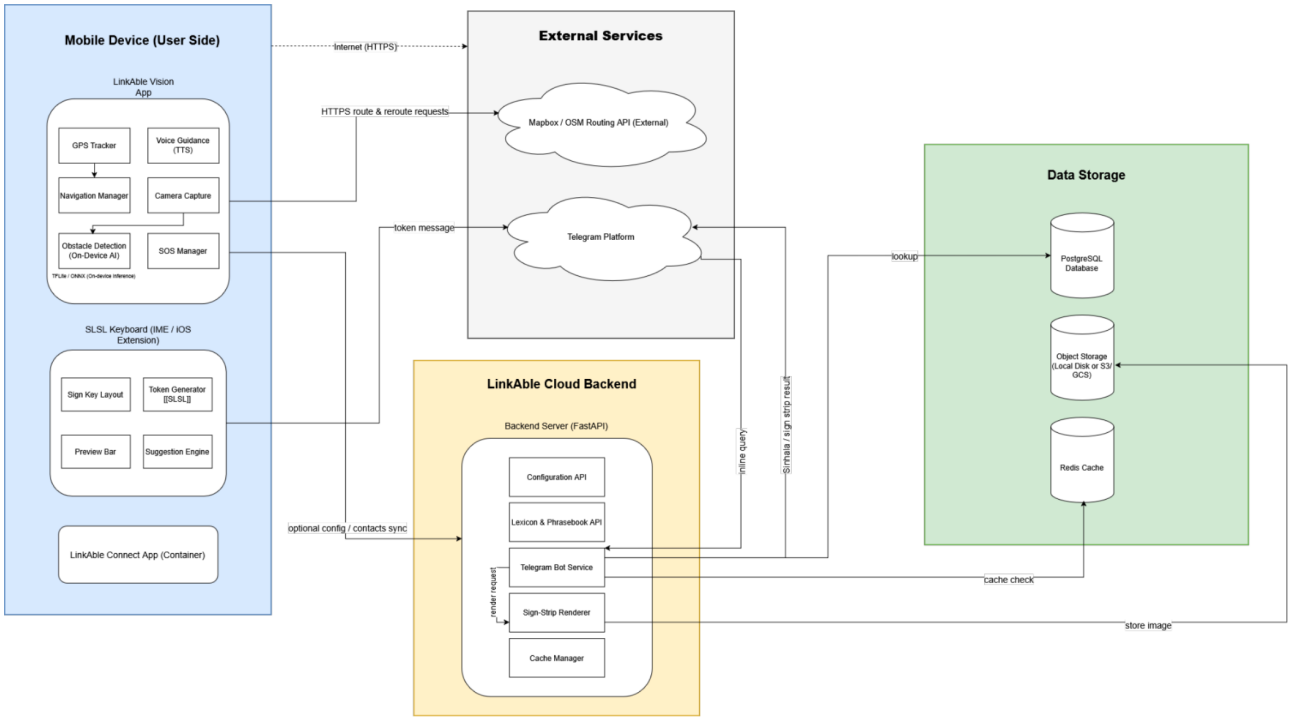

Figure 4. High-Level Architectural Diagram

5.5 Networking Diagram

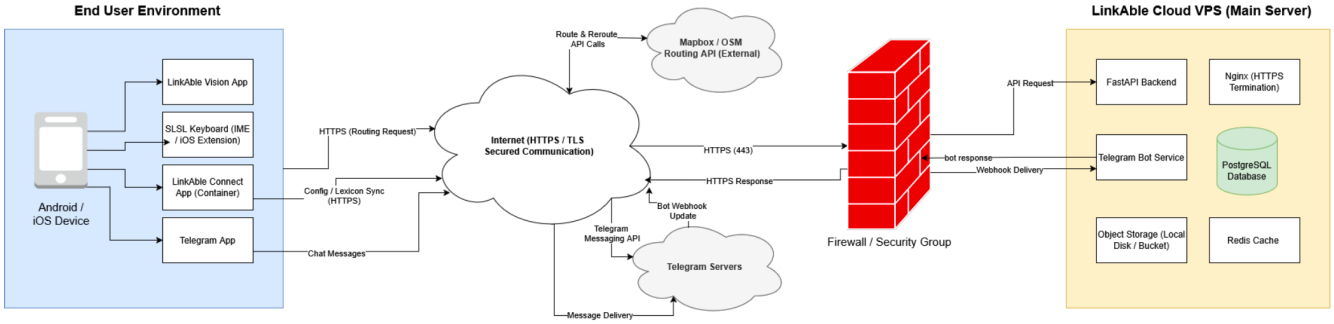

Figure 5. Networking Diagram

Chapter 06 – Development Tools & Technologies

6.1 Introduction

The chapter presents an overview of the technological resources which include tools and frameworks and programming languages and their associated technologies which all work together to develop two distinct LinkAble applications. The selected options deliver operational efficiency and capacity to function without internet access and predictable system performance and ability to conduct tests.

6.2 Overall engineering approach

Our team established a Mobile-first Reliability-first strategy which requires all essential functions to operate on user devices while maintaining minimal resource requirements for external systems and creating systems that allow complete performance tracking through testing. Vision requires on-device camera inference and alert decision logic that operates independently from network connections. Connect provides users with the ability to use keyboard typing and preview functions while working offline and they can choose to enable a bot that performs conversion and image rendering tasks.

6.3 Tools and technologies for LinkAble Vision

Mobile application layer

- React Native + TypeScript: Used for the shared application layer, state handling, and the obstacle alert decision engine. TypeScript improves maintainability with strong typing and enables unit tests for safety rules (cooldowns, prioritization, deduplication).
- App navigation framework: Organizes screens such as onboarding, navigation session, settings, and debug views.
- State management: A predictable state pattern coordinates navigation session state, permission state, and speech/alert queues.

Native mobile components

- iOS (Swift, AVFoundation): Captures camera frames efficiently, manages permissions, and runs the on-device ML interpreter.
- Android (Kotlin, CameraX-style pipeline): Provides camera frames, permission management, and on-device inference, using concurrency controls for frame sampling and backpressure.

On-device machine learning

- TensorFlow Lite (TFLite): Runs the obstacle detection model fully offline; selected for mobile efficiency and broad platform support.
- Model packaging: A YOLO-style detector exported to a mobile format is bundled in app assets. Pre-processing (resize/normalize) and post-processing (non-maximum suppression) are handled natively for speed.
- Decision logic and speech planning: A TypeScript alert engine converts detections into safe speech intents using thresholds, hysteresis, per-category cooldowns, rate limiting, and

priority handling. Scene summarization reduces repeated object naming in crowded environments.

Navigation and location

- Device services (GPS/compass): Provide location and heading for turn-by-turn walking guidance.
- Routing API integration: A routing provider supplies path computation and step instructions. The app treats routing as best effort and continues local obstacle awareness when routing updates fail.

6.4 Tools and technologies for LinkAble Connect

Keyboard input (mobile)

- Android IME (Kotlin): Implements a custom SLSL keyboard for token insertion into any text field, plus a lightweight preview/suggestion bar.
- iOS keyboard extension (Swift): Mirrors the SLSL keyboard on iOS while respecting extension limits (memory, execution time, restricted networking).
- On-device suggestion engine: A lightweight n-gram or frequency model proposes next signs; it runs offline and stays deterministic for the same input history.

Conversion and sharing

- Telegram inline bot (Python): Converts between sign tokens and Sinhala text and returns sign-strip images. Python is selected for fast development and strong libraries for parsing and rendering.
- Rendering stack (Pillow): Stitches sign icons into one or more strip images, splitting long sequences at defined boundaries. Deterministic rendering ensures identical inputs produce identical outputs.
- Service hardening: Strict token parsing, payload size limits, and rate limiting improve reliability. Unknown items are marked explicitly rather than guessed.

6.5 Collaboration, quality, and project tooling

Version control and teamwork

- Git + GitHub (or equivalent): Source control, branching, code reviews, and issue tracking.
- Interface contracts: Shared event/data contracts reduce integration friction between Vision and Connect modules.

Continuous integration was used to keep builds reproducible. A simple pipeline runs linting, TypeScript tests, and packaging steps for Android and iOS. Release artifacts are versioned so the team can compare performance across iterations. For the bot, a requirements lockfile and virtual environment ensure identical dependencies during deployment and testing. This reduces integration risk and supports faster interim demonstrations.

Testing

- Jest (TypeScript): Unit tests for the Vision alert engine and any shared utilities. Tests cover cooldown correctness, prioritization, deduplication, and summary message selection.

- Platform testing: Device/emulator testing validates keyboard behavior (cursor edits, token insertion, preview sync) and Vision behavior (permissions, camera stability, speech timing).
- Scenario scripts: Repeatable scenarios (crowded walkway, indoor obstacles, long sign message, unknown tokens) support consistent evaluation.

Build and debug

- Logging and metrics: Structured logs capture inference latency, FPS, dropped frames, alerts per minute, bot response times, and error types.
- Debug screens: In-app debug views show permission state, performance counters, and last emitted intents.

6.6 Summary table

Table 5. Technologies used in LinkAble

Area	LinkAble Vision	LinkAble Connect
Mobile UI/app layer	React Native + TypeScript	Container/settings (if used) + native keyboard UI
Native iOS	Swift + AVFoundation + TFLite	Swift keyboard extension
Native Android	Kotlin + camera pipeline + TFLite	Kotlin IME keyboard
AI/ML	On-device TFLite object detector	On-device n-gram/frequency suggestions
External integration	Routing API, device GPS/compass	Telegram inline bot
Service stack	Optional lightweight endpoints	Python bot + Pillow rendering
Testing	Jest + device scenarios	Keyboard tests + bot tests

Chapter 07 – Implementation Progress

7.1 Introduction

This chapter reports what has been implemented so far for the LinkAble project and how the current build aligns with the defined requirements. LinkAble consists of two distinct software programs which include LinkAble Vision that provides navigation and obstacle detection for blind and low-vision users and LinkAble Connect which enables users to create Sri Lankan Sign Language via a sign keyboard that works with Telegram for sharing purposes. Implementation progress is presented for each application, followed by integration status, testing progress, challenges, and the plan for the remaining work.

7.2 Overall Progress Summary

At interim stage, the project has moved beyond paper design into working prototypes and partially integrated modules. Vision progress focuses on the real-time obstacle detection subsystem and safe speech alert generation while maintaining offline resilience. Connect progress focuses on the keyboard input pipeline (deterministic sign tokens + preview) and the Telegram inline bot's conversion and sign-strip rendering.

Table 6. High-level progress by application

Component area	Current status	Evidence type available
Vision: navigation session flow	In progress	UI flow + route demo (partial)
Vision: on-device obstacle detection	Implemented (core)	Live inference + metrics logs
Vision: non-spam alert logic	Implemented (core)	Unit tests + live demos
Vision: SOS flow	In progress	UI + messaging hooks (partial)
Connect: Android IME keyboard	Implemented (core)	APK build + typing demo
Connect: iOS keyboard extension	In progress	Early build / scaffolding
Connect: token format + preview	Implemented (core)	Demo + edit-sync checks
Connect: Telegram inline bot + rendering	Implemented (core)	Bot project + image outputs
Shared: content assets (lexicon/phrasebook)	In progress	Data format + version plan
Shared: testing + evaluation tooling	Implemented (partial)	Logs, debug screens, test runs

7.3 LinkAble Vision – Implementation Progress

7.3.1 Implemented: Camera pipeline and controlled sampling

The Vision obstacle detection feature required a stable camera pipeline before model inference could be useful. The build now includes a controlled frame sampling strategy so the app does not attempt to process every camera frame. Sampling is adjustable (target FPS), and the system enforces backpressure: if inference is still running, new frames are dropped rather than queued.

This prevents alert latency from increasing as the device gets busy and ensures the user hears timely warnings rather than delayed speech.

Permissions were implemented as part of the pipeline. Camera permission state is tracked and emitted to the app so the UI can display clear recovery guidance (enable permission, retry). This prevents silent failure where the model appears broken but the actual cause is missing permission.

7.3.2 Implemented: On-device inference integration

The current build integrates TensorFlow Lite inference on both iOS and Android via native modules. The model is packaged with the app so obstacle detection works offline and does not rely on server latency or connectivity. The inference loop includes input resizing/normalization and output decoding into bounding boxes and confidence scores. Post-processing includes non-maximum suppression so that overlapping detections do not produce multiple redundant alerts.

A key progress achievement is that detections are emitted in a standardized format to the TypeScript layer, allowing the alert logic to remain identical across platforms. This improves consistency and makes the system testable even when switching devices.

7.3.3 Implemented: Alert engine (non-spam, safety-first speech)

Obstacle detection alone is not enough for a blind navigation assistant; the system must avoid overwhelming the user. The alert engine is implemented as a TypeScript decision layer that converts raw detections into safe speech intents. It includes:

- Global minimum gap between spoken alerts
- Per-category cooldowns to prevent repetitive messages
- Deduplication (avoid repeating the same obstacle repeatedly)
- Hysteresis/stability gating (require consistent detection over multiple frames)
- Priority handling so urgent hazards can override lower-priority messages
- Rate limiting so speech does not become continuous noise

To address realistic crowded scenes, the implementation includes a summarization layer. Instead of repeating “person, person, person,” the engine produces scene-level messages such as “crowd ahead” or “path blocked ahead,” optionally expanding detail based on sensitivity settings. This makes the feature usable in public environments while maintaining awareness.

7.3.4 Implemented: Sensitivity modes and localization-ready outputs

The interim build includes adjustable sensitivity modes (e.g., Conservative/Balanced/Talkative). These modes tune thresholds, cooldown durations, and maximum alerts per minute. This gives the user control over verbosity and supports different mobility confidence levels.

Messages are emitted as localization-ready keys rather than hardcoded text-only output, enabling multi-language support later (English/Sinhala/Tamil). Even if translation quality is improved in later stages, the structure ensures the system is ready for localization without rewriting the alert logic.

```typescript
export type ObstacleCategory =
  | 'PERSON'
  | 'VEHICLE'
  | 'STAIRS_UP'
  | 'STAIRS_DOWN'
  | 'POLE'
  | 'WALL'
  | 'CURB'
  | 'BIKE'
  | 'DOG'
  | 'UNKNOWN';

/**
 * Normalized bounding box coordinates (0.0 to 1.0).
 * Origin at top-left corner of frame.
 *
 * @property x - Normalized X coordinate of top-left corner
 * @property y - Normalized Y coordinate of top-left corner
 * @property w - Normalized width
 * @property h - Normalized height
 */
export interface BoundingBoxNormalized {
  readonly x: number;
  readonly y: number;
  readonly w: number;
  readonly h: number;
}
```

```typescript
export type AlertPriority = 'HIGH' | 'MED' | 'LOW';

/**
 * Spatial position hint for obstacle location.
 */
export type SpatialPosition = 'LEFT' | 'CENTER' | 'RIGHT';

/**
 * Alert event metadata for context-rich notifications.
 */
export interface AlertMetadata {
  readonly confidence?: number;
  readonly position?: SpatialPosition;
  readonly distance?: number; // Future: depth estimation
}

/**
 * User-facing alert event generated by alert engine.
 * Immutable event ready for TTS, haptics, or UI notification.
 */
export interface ObstacleAlertEvent {
  readonly id: string;
  readonly category: ObstacleCategory;
  readonly priority: AlertPriority;
  readonly messageKey: string; // i18n key for TTS
  readonly fallbackEnglish: string; // Fallback if i18n unavailable
  readonly timestampMs: number;
  readonly ttlMs: number; // Time-to-live for deduplication
  readonly meta?: AlertMetadata;
```

Figure 6. Code snippets of categorizing obstacles

7.3.5 Implemented: Debug and evaluation tooling

A debug overlay screen and structured metrics logging are implemented to support measurable evidence. The system logs:

- Target vs actual FPS
- Inference latency
- Dropped frame count
- Alerts per minute
- Suppression counts (cooldown hits, rate-limit hits)
- Recent scene flags and alert history

This tooling strengthens interim evaluation by producing quantitative results and allowing quick diagnosis when performance differs across devices.

7.3.6 In progress: Navigation session and SOS integration

Navigation features and SOS are being integrated into the same session model. Work in progress includes mapping route steps to speech, detecting deviation for rerouting, and integrating SOS location sharing and confirmation messaging. The obstacle subsystem is already designed to keep running even when routing updates cannot be fetched, supporting safe degradation under poor connectivity.

7.4 LinkAble Connect – Implementation Progress

7.4.1 Implemented: Android keyboard (IME) core workflow

Connect progress includes a working Android IME build distributed as an APK. The keyboard supports sign-key input and inserts deterministic sign tokens into text fields. The keyboard design focuses on speed and consistency: sign keys map to known sign IDs, producing the same token format every time.

The offline usability goal is already satisfied at the typing level: the keyboard can be used without internet, and token insertion does not depend on any remote service.

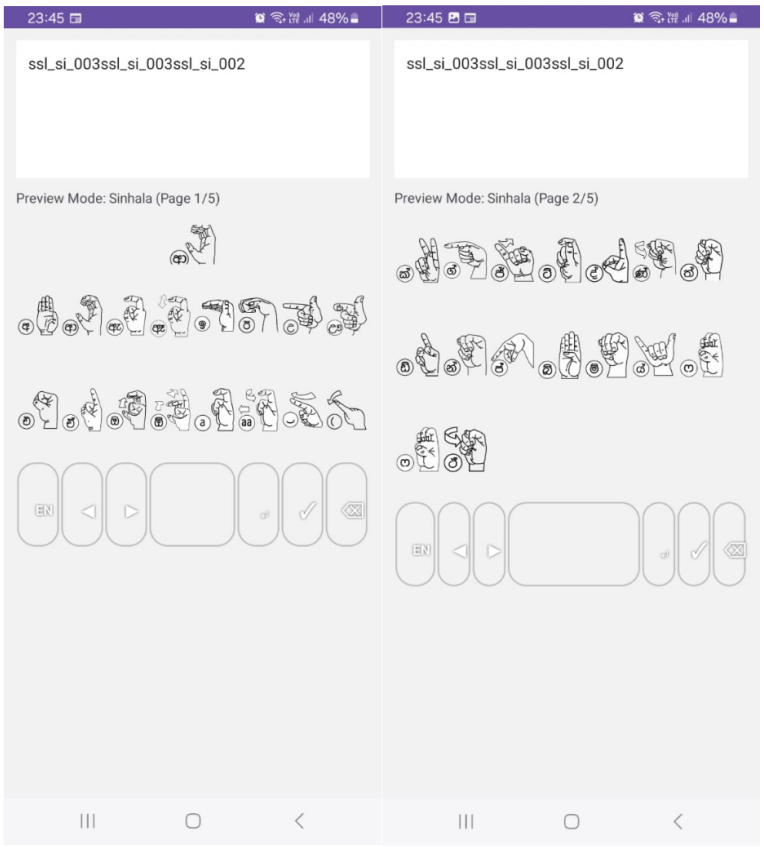

Figure 7. Current status of the Linkable connect with a new addition of preview bar

7.4.2 Implemented: Token format enforcement and local preview

The keyboard implements strict token formatting rules so output remains parsable and safe for conversion. A local preview bar shows the user's sign sequence as icons, reducing ambiguity compared to plain token strings. The preview logic has been validated under common edits such as backspace, insertion, and retyping.

This deterministic approach is important for correctness: it enables repeatable tests and prevents guessing during conversion. It also improves user trust, because the same sign sequence always yields the same rendered output when the mapping is known.

7.4.3 Implemented: On-device suggestion concept (baseline)

Connect includes a baseline approach for next-sign suggestions using lightweight frequency patterns (n-gram style). At interim stage, suggestions prioritize feasibility: low compute cost, offline support, and deterministic results. The suggestion bar is treated as an assistive enhancement rather than a dependency—typing remains fully functional without accepting any suggestions.

7.4.4 Implemented: Telegram inline bot and sign-strip rendering

A Telegram inline bot implementation is available as a Python project ZIP. It supports parsing token sequences and generating sign-strip images by stitching sign icons in order. If a message becomes long, output is split into multiple strips to preserve readability and prevent oversized images.

The bot's design focuses on controlled conversion rather than smart translation. Unknown inputs are treated explicitly: if a token or word is not in the mapping, the bot marks it as unknown instead of guessing. This is critical for sign communication because incorrect signs can alter meaning.

7.4.5 In progress: iOS keyboard extension and production hardening

iOS keyboard extension work is ongoing. iOS constraints (memory limits and restricted background processing) require careful UI design and minimal processing inside the extension. Current progress focuses on keyboard layout parity and safe token insertion. Bot hardening (payload limits, rate limiting, strict validation) is planned to strengthen operational stability before final release.

7.5 Integration Progress and Interfaces

Although Vision and Connect are separate applications, they share certain assets and standards: lexicon/phrasebook versioning, language configuration patterns, and a consistent approach to logging, error messages, and consent-based analytics. Integration progress is focused on making these shared elements consistent without coupling the apps tightly.

7.6 Testing and Validation Progress

Testing has been approached in three layers:

1) Unit tests (logic correctness): Vision alert gating and summarization rules are tested using scripted detections. Connect parsing rules and rendering split rules are tested using sample token sequences.

2) Scenario tests (end-to-end): For Vision, scenarios include crowded scenes, repeated objects, and changing motion. For Connect, scenarios include long messages, unknown tokens, and frequent edits while typing.

3) Performance checks (device-based): Vision measures inference latency and alert rates; Connect measures keyboard responsiveness and bot response times.

The project now has enough instrumentation to support evidence-based interim evaluation rather than purely descriptive claims.

7.7 Challenges Encountered and Solutions Applied

- Real-time performance vs accuracy (Vision): Continuous frame processing created latency risk. The solution was controlled sampling and backpressure (drop frames when busy) plus speech rate limiting.
- Speech overload in crowded scenes (Vision): Raw object detection created repetitive alerts. The solution was scene summarization and sensitivity modes.
- OS constraints for keyboards (Connect): Keyboard extensions cannot behave like full apps. The solution is to keep the keyboard lightweight and to move heavy conversion/rendering to the Telegram bot.
- Determinism and safety (Connect): Guessing unknown signs is risky. The solution is strict token formats and explicit unknown handling.
- Integration overhead: Separate apps still need consistent shared assets. The solution is to define clean schemas and versioning early, while keeping deployment independent.

7.8 Remaining Work Before Final Delivery

Key remaining work is split evenly across apps:

Vision remaining items: complete robust navigation session behavior, rerouting logic, SOS end-to-end verification, and additional field testing for alert usefulness across environments and devices.

Connect remaining items: finalize iOS keyboard extension parity, strengthen bot hardening and deployment stability, expand lexicon coverage, and formalize test cases for token parsing and rendering determinism.

7.9 Interim Conclusion

At this interim stage, LinkAble has working cores for both applications: Vision's on-device obstacle detection plus non-spam alert engine, and Connect's deterministic keyboard workflow plus bot-based sign-strip rendering. The remaining work focuses on completing feature completeness (navigation + SOS for Vision, iOS keyboard maturity for Connect), strengthening shared assets, and expanding evaluation so the final report can present measurable evidence of usability, performance, and reliability.

Table 7. Integration status for shared elements

Shared element	Purpose	Current status	Next steps
Lexicon/phrasebook structure	Consistent mapping for signs and labels	In progress	Finalize schema + version tags
Localization keys	Multi-language readiness	Partial	Expand dictionaries + UI toggles
Logging/metrics format	Comparable evaluation evidence	Partial	Standardize event names + export
Consent and privacy settings	Safe analytics and sharing	In progress	Add opt-in controls and disclaimers

Chapter 08 – Discussion

The interim stage shows that LinkAble development progresses through two distinct applications which both follow a unified approach to quality. The main outcome for LinkAble Vision must achieve on-device obstacle detection operations through an experience which meets actual mobility requirements of delivering speech at appropriate times, keeping background sounds to a minimum, and allowing users to behave safely during periods of routing updates and connectivity problems. The alert engine design shows assistive usability thinking rather than raw object recognition, because crowded environments can easily overwhelm a user if every detection becomes spoken output. The system development includes backpressure and cooldown rules and prioritization and summarization processes which show a design approach that builds system stability and safety trustworthiness needed for critical safety situations.

The project establishes its core communication framework through sign composition in LinkAble Connect because the interim build demonstrates that users require fast and predictable performance which matches their actual messaging behaviors. The Android keyboard and bot rendering pipeline demonstrate an end-to-end path from sign selection to shareable output without depending on fragile guessing. Deterministic token formatting and explicit unknown handling are especially significant because misinterpretation in sign communication can change meaning, harm user confidence, and reduce adoption. The user control system allows two processes through their specific functions: users can share documents through Telegram inline bots which need their active choice or they can work offline through lightweight keyboards which their design requires.

Across both apps, the main discussion point is integration without confusion: shared assets such as lexicon/phrasebook and localization structures must be consistent, but the apps must remain independently testable and deployable. The next stage should prioritize completion of Vision navigation + SOS flows, parity for the iOS Connect keyboard, and stronger evidence through structured evaluations (latency, alert rates, bot response times, and user feedback). This will ensure the final delivery is not only feature-complete but demonstrably reliable.

References

1. American Speech-Language-Hearing Association (ASHA). (2023). *Augmentative and Alternative Communication (AAC) Overview.* Available at: https://www.asha.org (Accessed: 15 October 2025).

2. Crews, J.E. and Campbell, V.A. (2021). Vision Impairment and Risk of Falls, Isolation, and Unemployment among Older Adults. American Journal of Public Health, 111(3), pp. 412–418.

3. Google Research. (2023). *Advances in Speech-to-Text and Real-Time Transcription for Accessibility.* Google AI Blog. Available at: https://ai.googleblog.com (Accessed: 16 October 2025).

4. Khan, M., Li, Z., and Hossain, A. (2024). Deep Learning for Real-Time Object Detection on Mobile Devices. ACM Transactions on Interactive Intelligent Systems, 12(4), pp. 210–228.

5. Microsoft. (2022). *Soundscape: Audio Navigation for the Blind.* Microsoft Research. Available at: https://www.microsoft.com/research (Accessed: 20 October 2025).

6. OpenStreetMap Foundation. (2024). *Accessible Mapping Initiatives in Developing Regions. OSM Annual Report 2024.*Available at: https://www.openstreetmap.org (Accessed: 23 October 2025).

7. Shield, B. (2023). Hearing Loss – Numbers and Global Impact. British Medical Bulletin, 138(1), pp. 1–10.

8. SignAll Technologies. (2023). Automated Sign Language Recognition Using Deep Neural Networks. IEEE Access, 11, pp. 22560–22574.

9. Tager-Flusberg, H. and Kasari, C. (2022). Nonverbal Individuals with Autism Spectrum Disorder: Research Review and Future Directions. Autism Research, 15(2), pp. 191–206.

10. Wijesinghe, R., Perera, T., and De Silva, N. (2024). Challenges in Localizing AI Accessibility Tools in South Asia. Journal of Assistive Computing, 18(2), pp. 45– 59.

Team Plan & Responsibility Matrix

Phase 1 – Planning & Setup: Finalize scope, requirements, and interfaces; set up repos, branching, coding standards, and shared data schemas (lexicon/phrasebook, tokens, events).
Phase 2 – Core Development: Build Vision modules (routing, obstacle AI, audio, SOS, permissions) and Connect modules (Android IME, iOS extension scaffolding, preview/sign suggestions, Telegram bot rendering).
Phase 3 – Integration & Stabilization: Integrate modules using agreed contracts, fix cross-module issues, harden error handling, and ensure offline-safe behavior for both apps.
Phase 4 – Testing & Release: Execute unit + scenario tests, collect performance metrics, prepare builds, write report evidence, and complete final packaging/submission.

Table 8. Responsibility Matrix

Student ID	Student Name	Responsibilities (Vision + Connect)
10967100	SAP Prabhashana	Vision: Obstacle Detection AI Model (On-device CV) + alert logic Connect: SLSL Linguistics and Keyboard/IME Core engineering
10967081	RRI Dissanayaka	Vision: Vision Navigation & Routing (Mapbox / OSM) Connect: Preview sign feature; Messaging integration (Telegram)
10967122	YUS De Yapa	Vision: Audio Layer (TTS + Sound Cues) + Voice UX Connect: Preview sign feature; Data Analytics
10967104	Rubasinghe D D	Vision: SOS + Emergency Contacts + Secure Storage Connect: Android app backend
10967063	NPMMD Pathirana	Vision: Vision UI/UX + Accessibility & Settings Connect: Android app backend; Backend and APIs
10967076	AVJA Amarasekera	Vision: Permissions, Reliability, Offline/Failure Handling Connect: UI/UX; Data Analytics
10967111	PGT Sewmini	Vision: Testing, Evaluation, and Release Packaging Connect: iOS app backend